ECEN 749 Lab 7 - Report

Yashas Bharadwaj
UIN: 635009332; Section: 601

Introduction

This lab focuses on using built-in kernel drivers, a departure from the previous labs' use of loadable kernel modules. It requires the creation of a C application file to utilize the device driver and validate multiplication, similar to the preceding lab. The kernel module binary and the executable for the new application C file are the only components mounted onto the SD card for deployment on the Zybo Z7-10 board.

Procedure

1. Setup and Build: Execute petalinux_build. Unzip the Linux source from ECEN449 into the PetaLinux project (linux_boot).
2. Driver Integration: Create drivers/multiplier_driver in the Linux source. Place multiplier.c, xparameters.h, and xparameters_ps.h inside. Create and configure Makefile and Kconfig in multiplier_driver, and modify the main "Device Drivers" Makefile and Kconfig to include it.
3. Kernel Build: Link the modified Linux source to the PetaLinux project via petalinux-config (external kernel source). Run petalinux-config -c kernel to select the multiplier driver as a built-in module. Execute petalinux-build.
4. Deployment and Test: Copy image.ub, BOOT.bin, boot.scr, and devtest to the SD card. On the Zybo via picocom, mount the SD card: mount /dev/mmcblk0p1 /mnt/. Note the mknod /dev/MULTIPLIER C 246 0 message. Navigate to /mnt and run /devtest to display the required output.

Results

The Linux boot process was observed on the picocom console, prompting for a user login and password. To validate the kernel module's functionality, root access was obtained by typing sudo su. Subsequently, the necessary outputs were made visible by executing the mknod commands.

Initially, the image.ub file was approximately 18 MB. The file size was successfully reduced to 16 MB after rebuilding the project with the Network device, Multimedia, and Sound card support disabled.

Conclusion

This lab successfully demonstrated the process of creating internal built-in modules within the Linux source code, a contrast to the previously used method of loadable kernel modules. It provided valuable insight into integrating a multiplier driver directly into the existing Linux source. The menuconfig interface was utilized to select and compile the new multiplier driver

and its necessary modules. Finally, the built-in kernel module's functionality was validated in a Petalinux system using the picocom console to view the output and by developing corresponding application C programs.

Questions

a) Loadable kernel modules:
   - Advantages: It can add and remove features without rebooting, saves memory by loading only required modules.
   - Disadvantages: It is slightly slower than the built-in modules.

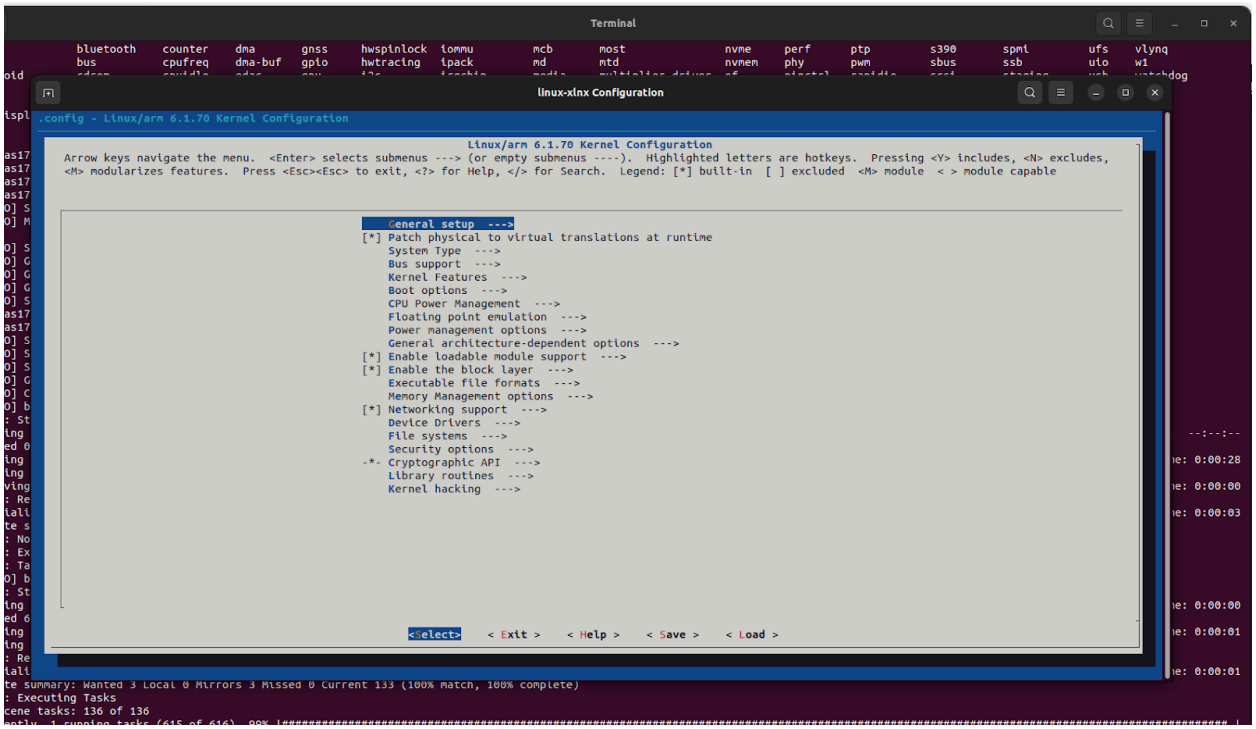
b) Built-in modules:
   - Advantages: It is always available at boot and it is slightly faster because they are part of the kernel.
   - Disadvantages: Can’t add or remove without rebuilding the kernel and uses more memory

Appendix

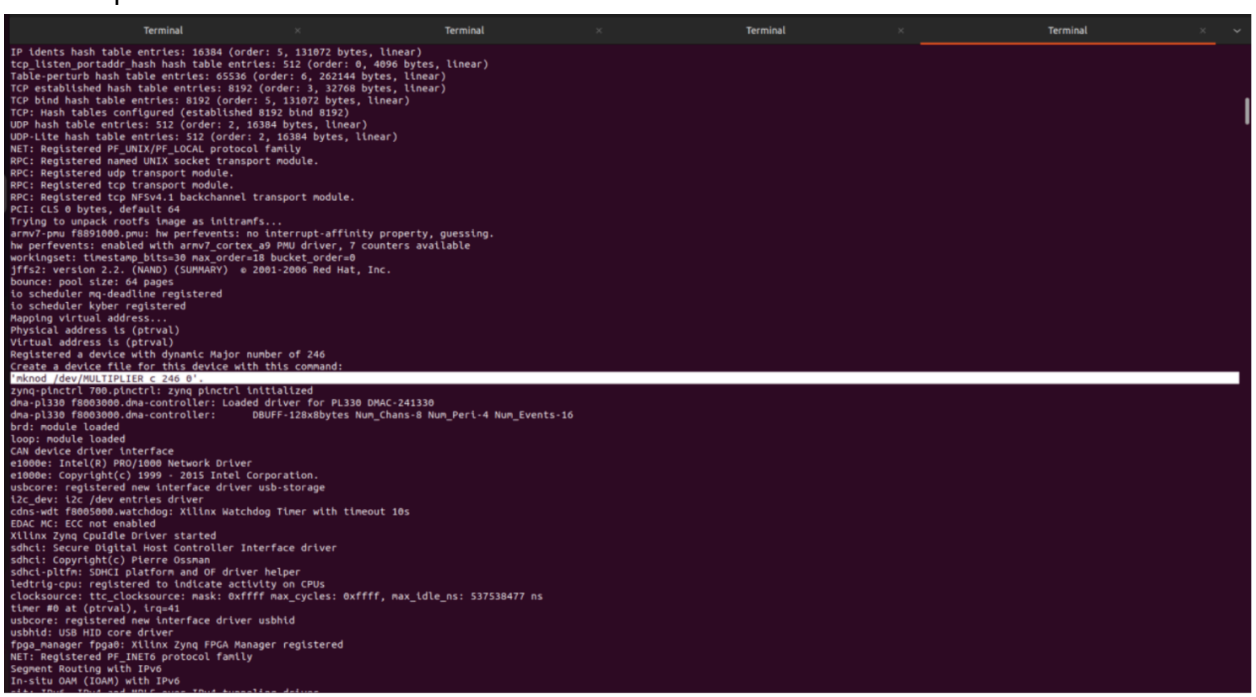
Picocom console:

```
Result Correct!

Writing 2 to register 0
Writing 4 to register 0
Kernel Buffer[0] = 2
Kernel Buffer[1] = 4
Kernel Buffer[2] = 8
2 * 4 = 8
Result Correct!

Writing 2 to register 0
Writing 5 to register 0
Kernel Buffer[0] = 2
Kernel Buffer[1] = 5
Kernel Buffer[2] = 10
2 * 5 = 10
Result Correct!

Writing 2 to register 0
Writing 6 to register 0
Kernel Buffer[0] = 2
Kernel Buffer[1] = 6
Kernel Buffer[2] = 12
2 * 6 = 12
Result Correct!

Writing 2 to register 0
Writing 7 to register 0
Kernel Buffer[0] = 2
Kernel Buffer[1] = 7
Kernel Buffer[2] = 14
2 * 7 = 14
Result Correct!

Writing 2 to register 0
Writing 8 to register 0
Kernel Buffer[0] = 2
Kernel Buffer[1] = 8
Kernel Buffer[2] = 16
2 * 8 = 16
Result Correct!

Writing 2 to register 0
Writing 9 to register 0
Kernel Buffer[0] = 2
Kernel Buffer[1] = 9
Kernel Buffer[2] = 18
2 * 9 = 18
Result Correct!
```

MenuConfig:



Boot Sequence:



Original Image Size:

```
Terminal
PetaLinux environment set to '/tmp/petalinux_yashas1729/petalinux_install'
WARNING: /bin/sh is not bash!
bash is PetaLinux recommended shell. Please set your default shell to bash.
INFO: Checking free disk space
INFO: Checking installed tools
INFO: Checking installed development libraries
INFO: Checking network and other services
WARNING: No tftp server found - please refer to "UG1144 2023.1 PetaLinux Tools Documentation Reference Guide" for its impact and solution
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729$ cd linux_boot_yashas1729/
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729$ petalinux-build
[INFO] Sourcing buildtools
[INFO] Building project
[INFO] Silentconfig project
[INFO] Silentconfig rootfs
[INFO] Generating workspace directory
ERROR: Unable to connect to bitbake server, or start one (server startup failures would be in bitbake-cookerdaemon.log).
ERROR: Unable to connect to bitbake server, or start one (server startup failures would be in bitbake-cookerdaemon.log).
ERROR: Failed to create workspace directory
ERROR: Failed to build project. Check the /tmp/petalinux_yashas1729/linux_boot_yashas1729/build/build.log file for more details...
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729$ ls
build  components  images  project-spec
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729$ cd components/
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/components$ cd layers
bash: cd: layers: No such file or directory
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/components$ ls
ext_sources  plnx_workspace  yocto
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/components$ cd ext_sources/
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/components/ext_sources$ ls
linux-xlnx-xlnx_rebase_v6.1_LTS
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/components/ext_sources$ cd linux-xlnx-xlnx_rebase_v6.1_LTS/
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/components/ext_sources/linux-xlnx-xlnx_rebase_v6.1_LTS$ ls
arch   certs    CREDITS  Documentation  fs      init      ipc     Kconfig  lib       MAINTAINERS  mm   oe-logs     README  samples  security  tools  virt
block  COPYING  crypto   drivers        include io_uring  Kbuild  kernel   LICENSES  Makefile     net  oe-workdir  rust    scripts  sound     usr
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/components/ext_sources/linux-xlnx-xlnx_rebase_v6.1_LTS$ cd drivers/
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/components/ext_sources/linux-xlnx-xlnx_rebase_v6.1_LTS/drivers$ ls
accessibility  bluetooth    cpufreq  edac      greybus     idle          leds       mfd              nubus    peci      ptp         sbus       staging      vdpa      zorro
acpi           bus          cpuidle  eisa      hid         iio           macintosh  misc             nvdimm   perf      pwm         scsi       target       vfio
amba           cdrom        crypto   extcon    hsi         infiniband    mailbox    mmc              nvme     phy       rapidio     sh         tc           vhost
android        cdx          cxl      firewire  hte         input         Makefile   most             nvmem    pinctrl   ras         siox       tee          video
ata            char         dax      firmware  hv          interconnect  mcb        mtd              of       platform  regulator   slimbus    thermal      virt
atm            clk          dca      fpga      hwmon       iommu         md         multiplier_driver opp       pnp       remoteproc  soc        thunderbolt  virtio
auxdisplay     clocksource  devfreq  fsi       hwspinlock  ipack         media      mux              parisc   power     reset       soundwire  tty          vlynq
base           comedi       dio      gnss      hwtracing   irqchip       memory     net              parport  powercap  rpmsg       spi        ufs          w1
bcma           connector    dma      gpio      i2c         isdn          memstick   nfc              pci      pps       rtc         spmi       uio          watchdog
block          counter      dma-buf  gpu       i3c         Kconfig       message    ntb              pcmcia   ps3       s390        ssb        usb          xen
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/components/ext_sources/linux-xlnx-xlnx_rebase_v6.1_LTS/drivers$ cd multiplier_driver/
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/components/ext_sources/linux-xlnx-xlnx_rebase_v6.1_LTS/drivers/multiplier_driver$ ls
Kconfig  Makefile  multiplier.c  xparameters.h  xparameters_ps.h
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/components/ext_sources/linux-xlnx-xlnx_rebase_v6.1_LTS/drivers/multiplier_driver$
```

Reduced Image Size:

```
Terminal                                                                    Terminal
Parsing recipes: 100% |##########################################################################################################| Time: 0:00:01
Parsing of 4344 .bb files complete (4339 cached, 5 parsed). 6275 targets, 357 skipped, 1 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 100% |#######################################################################################################| Time: 0:00:13
Sstate summary: Wanted 313 Local 285 Mirrors 6 Missed 22 Current 1228 (92% match, 98% complete)
WARNING: The cracklib-native:do_fetch sig is computed to be 9dc70ff8344d10501b14d2918c5da0d975de1fc55ffe7b42af17bb5c1733192e, but the sig is locked to 296da4145c53977745d54
f468ecac65c4921499a7abd5d76e8e45d074775f0a9 in SIGGEN_LOCKEDSIGS_t-x86-64
The cracklib:do_fetch sig is computed to be 9dc70ff8344d10501b14d2918c5da0d975de1fc55ffe7b42af17bb5c1733192e, but the sig is locked to 296da4145c53977745d54f468ecac65c49214
99a7abd5d76e8e45d074775f0a9 in SIGGEN_LOCKEDSIGS_t-cortexa9t2hf-neon
The cracklib-native:do_unpack sig is computed to be 8720bf6a6cea0d038864f7fe2485c354f02f6b87091eaa065f541ce375222d26, but the sig is locked to c16adc0993000ecbfab876afd386d
57bfd904e387a819d1c30f5b92952c7418e in SIGGEN_LOCKEDSIGS_t-x86-64
The cracklib:do_unpack sig is computed to be 75ca96c0affc2ddb4f38787421a9ebb9c2f5b2255cf00d3568500a42c44c32db, but the sig is locked to f19c0ff3b09b462be625586dbcd4a6481dc4
f9912901fc2667a011a7a411aeba in SIGGEN_LOCKEDSIGS_t-cortexa9t2hf-neon
The busybox:do_fetch sig is computed to be 16146b24d18a0700bfca13cdb5b9e2b8f48fd710c6a24ce7f779c35c33ad65c0, but the sig is locked to c2ac5f0c29da84190d0f46f4f48dcf67ef7255
138a4424d0a00004f7f614a942 in SIGGEN_LOCKEDSIGS_t-cortexa9t2hf-neon
The cracklib-native:do_populate_lic sig is computed to be fb8c48d2286f834773a054c9b6d1455ee3e16b98159b23a698ea99427a14c442, but the sig is locked to 059f83c7dcd22dda2a90313
2eead0fa4683ba47834aaf85c518aaadb3cda2fca in SIGGEN_LOCKEDSIGS_t-x86-64
The cracklib:do_populate_lic sig is computed to be 968af2fd6d5dafd0adde0c99d65a5b431f0dbe2db71b85b6e98df3bddc95462e, but the sig is locked to 6359632f75f1183510a4567460fc9f
f1a97ce4ad1d9b9a2052da3614dcbd311c in SIGGEN_LOCKEDSIGS_t-cortexa9t2hf-neon
Removing 19 stale sstate objects for arch zynq_generic_7z010: 100% |##############################################################| Time: 0:00:00
NOTE: Executing Tasks
NOTE: linux-xlnx: compiling from external source tree /tmp/petalinux_yashas1729/linux_boot_yashas1729/components/ext_sources/linux-xlnx-xlnx_rebase_v6.1_LTS
NOTE: Tasks Summary: Attempted 4135 tasks of which 4039 didn't need to be rerun and all succeeded.

Summary: There was 1 WARNING message.
INFO: Failed to copy built images to tftp dir: /tftpboot
[INFO] Successfully built project
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/components/ext_sources/linux-xlnx-xlnx_rebase_v6.1_LTS/drivers/multiplier_driver$ cd ~
yashas1729@zach-333-em3-20:~$ cd /tmp
yashas1729@zach-333-em3-20:/tmp$ ls
gdm3-config-err-T6bdhg                                          systemd-private-c9bd7c85f96247d180b7cccefcdd9cb0-power-profiles-daemon.service-NSQLj1
krb5cc_635009332                                                systemd-private-c9bd7c85f96247d180b7cccefcdd9cb0-switcheroo-control.service-HoxNtv
krb5ccmachine_ENGR.TAMU.EDU                                     systemd-private-c9bd7c85f96247d180b7cccefcdd9cb0-systemd-logind.service-xVxr9F
petalinux_yashas1729                                            systemd-private-c9bd7c85f96247d180b7cccefcdd9cb0-systemd-oomd.service-V19QR3
snap-private-tmp                                                systemd-private-c9bd7c85f96247d180b7cccefcdd9cb0-systemd-resolved.service-8IQ5af
systemd-private-c9bd7c85f96247d180b7cccefcdd9cb0-colord.service-0pahCf   systemd-private-c9bd7c85f96247d180b7cccefcdd9cb0-systemd-timesyncd.service-mXJxhG
systemd-private-c9bd7c85f96247d180b7cccefcdd9cb0-geoclue.service-AkprEO  systemd-private-c9bd7c85f96247d180b7cccefcdd9cb0-upower.service-iSydBH
systemd-private-c9bd7c85f96247d180b7cccefcdd9cb0-ModemManager.service-AqTSbC  tracker-extract-3-files.127
yashas1729@zach-333-em3-20:/tmp$ cd petalinux_yashas1729/
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729$ cd linux_boot_yashas1729/
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729$ ls
build  components  images  project-spec
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729$ cd images/
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/images$ cd linux/
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/images/linux$ ls
boot.scr  image.ub      rootfs.cpio     rootfs.cpio.gz.u-boot  rootfs.jffs2     rootfs.tar.gz  system.dtb  u-boot-dtb.bin  u-boot.elf  vmlinux  zynq_fsbl.elf
config    pxelinux.cfg  rootfs.cpio.gz  rootfs.ext4            rootfs.manifest  system.bit     u-boot.bin  u-boot-dtb.elf  uImage      zImage
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/images/linux$ ls -la image.ub
-rw-r--r-- 1 yashas1729 yashas1729 16514912 Dec  8 14:39 image.ub
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/images/linux$ ls -lh image.ub
-rw-r--r-- 1 yashas1729 yashas1729 16M Dec  8 14:39 image.ub
yashas1729@zach-333-em3-20:/tmp/petalinux_yashas1729/linux_boot_yashas1729/images/linux$
```

“Multiplier.c” code:

```
#include <linux/module.h>  /* Needed by all modules */
#include <linux/moduleparam.h>  /* Needed for module parameters */
#include <linux/kernel.h>  /* Needed for printk and KERN_* */
#include <linux/init.h>       /* Need for __init macros  */
#include <linux/fs.h>       /* Provides file ops structure */
#include <linux/sched.h>   /* Provides access to the "current" process task structure */
#include <asm/uaccess.h>   /* Provides utilities to bring user space data into kernel space.
Note, it is processor arch specific. */
#include <linux/slab.h>       /* kmalloc and kfree definitions */
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <asm/io.h>
#include "xparameters.h"
#include <linux/ioport.h>   // IO memory allocation
#include "my_chardev_mem.h"
#include "xparameters_ps.h"

// From xparameters.h
#define PHY_ADDR    XPAR_MULTIPLY_0_S00_AXI_BASEADDR // physical address of
multiplier

// Size of physical address range for multiply
#define MEMSIZE        XPAR_MULTIPLY_0_S00_AXI_HIGHADDR -
XPAR_MULTIPLY_0_S00_AXI_BASEADDR + 1

/* Some defines */
#define DEVICE_NAME    "multiplier"
#define BUF_LEN        80

/* Function prototypes, so we can setup the function pointers for dev
   file access correctly. */
int init_module(void);
void cleanup_module(void);
static int device_open(struct inode *, struct file *);
static int device_close(struct inode *, struct file *);
static ssize_t device_read(struct file *, char *, size_t, loff_t *);
static ssize_t device_write(struct file *, const char *, size_t, loff_t *);

// Virtual address pointing to multiplier
void* virt_addr;
```

```
/*
 * Global variables are declared as static, so are global but only
 * accessible within the file.
 */
static int Major;        /* Major number assigned to our device driver */
static int Device_Open = 0;    /* Flag to signify open device */


/* This structure defines the function pointers to our functions for
   opening, closing, reading and writing the device file.  There are
   lots of other pointers in this structure which we are not using,
   see the whole definition in linux/fs.h */
static struct file_operations fops = {
  .read = device_read,
  .write = device_write,
  .open = device_open,
  .release = device_close
};


static int __init my_init(void)
{

  printk(KERN_INFO "Mapping virtual address... \n");

  // map virtual address to multiplier physical address
  virt_addr = ioremap(PHY_ADDR, MEMSIZE);

  printk(KERN_INFO "Physical Address = %p \n", (void *)PHY_ADDR);
  printk(KERN_INFO "Virtual Address = %p \n", virt_addr);

  /* This function call registers a device and returns a major number
     associated with it.  Be wary, the device file could be accessed
     as soon as you register it, make sure anything you need (ie
     buffers ect) are setup _BEFORE_ you register the device.*/
  Major = register_chrdev(0, DEVICE_NAME, &fops);


  /* Negative values indicate a problem */
  if (Major < 0) {
    /* Make sure you release any other resources you've already
       grabbed if you get here so you don't leave the kernel in a
       broken state. */
    printk(KERN_ALERT "Registering char device failed with %d\n", Major);
```

```
    return Major;
  }

  printk(KERN_INFO "Registered a device with dynamic Major number of %d\n", Major);
  printk(KERN_INFO "Create a device file for this device with this command:\n'mknod /dev/%s c
%d 0'.\n", DEVICE_NAME, Major);

  return 0;        /* success */
}


static void __exit my_exit(void)
{
    // Unregister the device
    unregister_chrdev(Major, DEVICE_NAME);

    printk(KERN_ALERT "unmapping virtual address space... \n");
    iounmap((void *)virt_addr);
}


void my_cleanup(void)
{
  /*
   * Unregister the device
   */
  unregister_chrdev(Major, DEVICE_NAME);

  kfree(msg_bf_Ptr);            /* free our memory (note the ordering
                                here) */

}


/*
 * Called when a process tries to open the device file, like "cat
 * /dev/my_chardev".  Link to this function placed in file operations
 * structure for our device file.
 */
static int device_open(struct inode *inode, struct file *file)
{


/* In these case we are only allowing one process to hold the device
```

```
    file open at a time. */
  if (Device_Open)        /* Device_Open is my flag for the
                usage of the device file (definied
                in my_chardev.h)  */
   return -EBUSY;        /* Failure to open device is given
                back to the userland program. */

  Device_Open++;        /* Keeping the count of the device
                opens. */


   try_module_get(THIS_MODULE);    /* increment the module use count
                (make sure this is accurate or you
                won't be able to remove the module
                later. */

    cur_Ptr = msg_bf_Ptr;         /* set the ptr to the beginning of the
                              message */

   printk("Device opened successfully \n");
   return 0;
}


/*
 * Called when a process closes the device file.
 */
static int device_close(struct inode *inode, struct file *file)
{

   Device_Open--;        /* We're now ready for our next
                caller */

  /*
   * Decrement the usage count, or else once you opened the file,
   * you'll never get get rid of the module.
   */
   module_put(THIS_MODULE);

   printk("Device closed succesfully \n");
   return 0;
}
```

```
/*
 * Called when a process, which already opened the dev file, attempts to
 * read from it.
 */
static ssize_t device_read(struct file *filp,    /* see include/linux/fs.h   */
            char *buffer,    /* buffer to fill with data */
            size_t length,    /* length of the buffer     */
            loff_t * offset)
{
  /*
   * Number of bytes actually written to the buffer
   */
  int bytes_read = 0;

  // Allocating kernel buffer
  int * kernel_buffer = (int*)kmalloc(length * sizeof(int), GFP_KERNEL );

  // Read values from registers into kernel buffer
  kernel_buffer[0] = ioread32(virt_addr);
  kernel_buffer[1] = ioread32(virt_addr+4);
  kernel_buffer[2] = ioread32(virt_addr+8);

  printk("Kernel Buffer[0] = %d \n", kernel_buffer[0]);
  printk("Kernel Buffer[1] = %d \n", kernel_buffer[1]);
  printk("Kernel Buffer[2] = %d \n", kernel_buffer[2]);

  // Using char pointer
  char * kbuf = (char*)kernel_buffer;
  int i ;
  for( i = 0; i < length; i++ )
  {
    // Write / copy values from kernel space to user space buffer
    put_user(*(kbuf++), buffer++); // one char at a time...
    bytes_read++;
  }


  kfree(kernel_buffer);

  /*
   * Most read functions return the number of bytes put into the buffer
   */
  return bytes_read;
}
```

```
/*
 * Called when a process writes to dev file: echo "hi" > /dev/hello
 * Next time we'll make this one do something interesting.
 */
static ssize_t device_write(struct file *filp, const char *buff, size_t len, loff_t * off)
{
  int i;

  char * kernel_buffer = (char*)kmalloc((len+1)*sizeof(char), GFP_KERNEL);

  for( i = 0; i < len; i++ )
  {
      // Get / copy values from user space into kernel buffer
      get_user(kernel_buffer[i], buff+i);
  }

  kernel_buffer[len] = '\0';

  // Using int pointer
  int * int_buf = (int*)kernel_buffer;

  printk(KERN_INFO "Writing %d to register 0 \n", int_buf[0]);
  iowrite32(int_buf[0], virt_addr+0);

  printk(KERN_INFO "Writing %d to register 0 \n", int_buf[1]);
  iowrite32(int_buf[1], virt_addr+4);

  kfree(int_buf);

  /*
   * Again, return the number of input characters used
   */
  return i;
}


/* These define info that can be displayed by modinfo */
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Paul V. Gratz (and others)");
MODULE_DESCRIPTION("Module which creates a character device and allows user
interaction with it");
```

```
/* Here we define which functions we want to use for initialization
   and cleanup */
module_init(my_init);
module_exit(my_cleanup);
```